EXECUTION RECORD

VIDYUT / GROUP EE14 / PROTECT-90

Reproducibility checklist

The pipeline runs on a CPU in about four minutes once the window store exists. This checklist records the exact software versions, the expected directory layout, the execution order and the artefacts that each reported number depends on.

Group EE14 / AI For Engineers

Thapar Institute of Engineering and Technology

DOCUMENT 003 / REPRODUCIBILITY

NOVEMBER 2026

Contents

1 What is required 2
2 Dataset organisation 2
3 Execution sequence 2
4 Expected outputs 4
5 Verification steps 4
6 Limits of this checklist 4

SECTION 1 VIDYUT / EE14

What is required

Table 1. Software and hardware used for the reported results.

Item	Version / specification
Operating system	Windows 11 x86-64
Python	CPython 3.14.0
NumPy	2.3.5
pandas	3.1.0
scikit-learn	1.9.1
PyTorch	2.9.1, CPU build
joblib	1.6.0
Matplotlib	3.11.2
Hardware	16 logical cores, 16 GB RAM, no CUDA device

SECTION 2 VIDYUT / EE14

Dataset organisation

Download the record with DOI 10.5281/zenodo.21109169 and keep the metadata CSV beside the archive. The pipeline expects this layout in the project root:

```
hv_double_line_90kv_labels.csv 9,022 metadata rows hv_double_line_90kv_preprocessed_data.zip published archive
preprocessed/ created by preprocess.py hybrid_results/ created by train_hybrid.py
```

The archive holds one pickled DataFrame per episode, 6,400 rows by 49 columns. Extraction produces 9,022 files; the count is checked against the metadata before any training begins, and a mismatch aborts the run.

SECTION 3 VIDYUT / EE14

Execution sequence

1. Install the pinned environment: python -m pip install -r requirements.txt
2. Verify the environment and the dataset layout: python verify_cleanup.py
3. Build splits and window arrays: python preprocess.py
4. Check split integrity and window shapes: python verify_pipeline.py
5. Train and evaluate the three pipelines: python train_hybrid.py
6. Add the episode-level view: python evaluate_episode_level.py

Runtime is dominated by preprocessing, which re-reads every pickle and writes roughly 6.4 GB of binary window data in about eight minutes. CNN training takes about 185 s for 20 epochs on CPU, the descriptor forest fits in 33 s, and the hybrid adds about 9 s including embedding extraction. The full pipeline runs in about four minutes once the window store exists.

SECTION 4 VIDYUT / EE14

Expected outputs

Table 2. Artefacts and the check each one supports.

Artefact	Check
preprocessed/train|validation|test_episodes.csv	episode identifiers; no identifier appears twice
preprocessed/X_*.x.bin, y_*.bin	window arrays and labels; shapes [N, 256, 48]
preprocessed/norm_mean.npy, norm_std.npy	training statistics used for every split
hybrid_results/exp{A,B,C}_val_metrics.json	validation accuracy, macro-F1, confusion matrix, error count
hybrid_results/final_test_metrics.json	test metrics and the 22-error confusion matrix
episode_evaluation/episode_level_metrics.json	majority-vote accuracy over 1,354 episodes
window_signal_diagnostic.json	fault coverage before and after the sampler correction

SECTION 5 VIDYUT / EE14

Verification steps

Three checks should be run before any number is quoted. First, the split check: no episode identifier may appear in more than one split file. Second, the shape check: every window array must be [N, 256, 48] and every label array must be [N] with values in 0 to 3. Third, the reconstruction check in evaluate_episode_level.py, which re-runs the windowing function on each test episode and compares the result with the stored block; it aborts on any mismatch above 1e-5.

SECTION 6 VIDYUT / EE14

Limits of this checklist

A run on the full training split will not reproduce the reported numbers, because all three models train on the fixed 16,384-window subset. Reproduction requires that subset, seed 42 and the same package versions. No container image is provided, and no continuous integration is configured; the checklist describes what was executed, not a service.